Hit Rate Is Not Value: When Cache Metrics Mislead in Tiered LLM KV-Cache Persistence

Aditya Lawankar
lawankaraditya@gmail.com

Abstract

Large Language Model serving is bottlenecked by memory capacity for Key-Value (KV) caches. Multi-turn conversations force frequent cache eviction and costly recomputation (the "cold-start" problem). We build a three-tier KV cache persistence system (VRAM → NVMe → object storage) and evaluate six eviction policies (LRU, a recency heuristic, learned binary classification, static value-density with and without admission control, and a lifetime-normalized space-time density policy) under a protocol built to avoid common evaluation artifacts: virtual-clock trace replay, persona-driven synthetic workloads, tier-discounted value accounting, and paired comparisons across 10 seeds with 95% confidence intervals.

Our hypothesis was that a learned policy would beat LRU on cache hit rate. It does not: LRU wins hit rate on every constrained workload we test, with all paired deltas significant. The interesting result is what that verdict conceals, and why.

A cache hit is worth nothing unless the prefill it avoids exceeds the restore it costs, a threshold set by the break-even context length N^*, and above that threshold savings grow superlinearly in context length. Hit rate encodes neither effect, so it can rank policies backwards: on enterprise traffic the hit-rate winner delivers the *least* GPU time saved of any policy we test. Both effects are closed-form, which yields a decision rule that needs no predictor. Compute N^* per tier from the served architecture, decline to build tiers whose restore cannot be repaid, and admit entries only above threshold. Following that rule is worth more than any eviction policy we evaluated: deleting a tier that cannot pay raises delivered value 3.6× at identical capacity, where the best policy choice within a fixed hierarchy is worth 1.99× and, once provisioning is corrected, 1.15×.

We show when the two metrics agree, and confirm it three ways. They agree when cached items are near-uniform in value: on workloads far above N^*, and under Llama-2-70B economics where grouped-query attention places N^* at 12 tokens so that essentially every hit pays. They diverge when the workload straddles N^*. Switching architectures and switching hierarchies each isolate the same residual superlinearity, roughly 1.1×. N^* varies four orders of magnitude across architectures, and we validate the model against measured hardware.

A ground-truth ablation isolates the effect from every confound: two clairvoyant policies that both cache *perfectly*, 100.0% hit rate against 100.0%, a paired difference of zero on every seed, still deliver significantly different value, because once nothing need be evicted the remaining decision is which tier holds each entry, and hit rate cannot see it.

We also report two evaluation bugs of our own, because they moved the verdict in opposite directions and neither perturbed rankings at random. A wall-clock replay bug flattered the learned policy; a cost model containing no architecture at all, pricing prefill with constants whose linear/quadratic crossover sat an order of magnitude off any real transformer, flattered LRU, and invalidated a "static value density collapses" finding we had previously reported. Simple recency policies are insensitive to broken time, broken features, and broken value models alike; every richer policy is exposed to all three. Harness bugs therefore flatter whichever side of a comparison depends on the broken component, which we suggest is a systematic hazard for the learned-caching literature rather than an accident of our pipeline.

End-to-end validation with TinyLlama-1.1B measures a 19.1× TTFT gain on CPU, a T4 context-length sweep that crosses parity where predicted, extended on a long-context model to 9.1× at 6k tokens where cold prefill grows 14.8× against a 1.8× warm path, and an in-memory-control ablation in which the storage round-trip adds no output divergence beyond what any cache-resume path already incurs.

The system, the committed experiment data, and the scripts that regenerate every number and figure in this paper are available at https://github.com/aditya-lawankar/kv-cache-tier-persistence.

Keywords: Cache Replacement Algorithms, LLM Inference, Tiered Storage, KV Cache, ML Systems

1 Introduction

Serving a large language model proceeds in two phases. A *prefill* phase processes the entire prompt in parallel, populating the KV cache: the attention keys and values of every token at every layer. A *decode* phase then generates one token at a time against that cache. Prefill cost grows quadratically with context length, while the cache it produces grows only linearly (§2). In a multi-turn conversation, the cache is what makes turn boundaries cheap: as long as it survives in VRAM, a follow-up turn pays only for its new tokens.

VRAM has no room for idle sessions, however. A single Llama-2-7B session at its full 4k context holds 2.1 GB of KV state (§2.1), so an 80 GB A100 fits only ∼30 such sessions after weights, while a production service handles thousands of users whose think time between turns ranges from seconds to days. Serving engines such as vLLM therefore evict idle caches [7], and when the user returns, the entire conversation history must be re-prefilled from scratch. This is the *cold-start problem*: re-prefill wastes quadratically growing GPU compute and inflates time-to-first-token (TTFT) from tens of milliseconds to multiple seconds at long contexts [9].

Tiered persistence is the natural fix: demote idle caches down a storage hierarchy (VRAM → NVMe → object storage) and restore them on return, paying $O(N)$ bytes moved instead of $O(N^2)$ FLOPs recomputed. Production systems have begun doing exactly this [5, 13, 11]. But persistence does not eliminate the eviction problem; it relocates it. Every tier is still finite, so some policy must decide which sessions each tier retains and which fall out of the hierarchy entirely.

That policy decision looks like fertile ground for learning. Sessions are far from interchangeable: they differ by orders of magnitude in size, their recompute cost grows quadratically where their footprint grows linearly, and whether a user will return appears predictable from history (time of day, past return behavior, conversation length). LRU sees none of this. We therefore hypothesized that a learned policy, which estimates each session's probability of resumption and evicts the least promising, would beat LRU on cache hit rate.

The data falsified this hypothesis, and diagnosing why became the contribution. LRU wins hit rate on every constrained workload, and no amount of feature engineering closed the gap. But the diagnosis did not end there, because the metric turned out to be the problem. Scored by GPU time saved, the same LRU that wins every hit-rate comparison delivers the *least* value of any policy we tested on enterprise traffic, and policies that lose to it by 11–21 hit-rate points save up to twice as much compute (§9.3).

The reason is that hit rate assumes every hit is worth the same, and in KV caching they are not, in two distinct ways. A hit is worth *nothing* unless the prefill it avoids exceeds the restore it costs, a threshold given by the break-even context length N^* (§11); and above that threshold, savings grow superlinearly with context length, so a few large sessions can outweigh many small ones. A policy optimizing hit rate maximizes the count of hits while ignoring both effects, which is why it can win that metric and lose the one that pays for hardware.

A ground-truth ablation bounds the diagnosis: replaying the same traces with policies that read the future separates "the objective is wrong" from "the predictor is weak," and shows that even clairvoyant policies with identical, perfect hit rates differ measurably in value (§9.6).

Finally, we report a methodological result. Reaching the conclusions above required correcting our own evaluation twice, and the two corrections moved the verdict in opposite directions: a wall-clock replay bug had flattered the learned policy, and a cost model with no architecture in it had flattered LRU, at one point supporting a "static value density collapses" finding that did not survive (§9.1). The asymmetry is structural rather than accidental. LRU consumes one input; every richer policy consumes more, and is exposed to defects in each. Harness bugs therefore flatter whichever side of a comparison depends on the broken component, which makes baseline-versus-learned comparisons sensitive to evaluation fidelity.

This paper makes the following contributions:

1. A three-tier KV cache persistence system with pluggable eviction backends (LRU, heuristic, learned classification, value-density, space-time density) and an evaluation protocol designed to remove common artifacts: virtual-clock replay, persona-driven workloads, replay of the Azure LLM inference production trace, tier-discounted value accounting, and paired multi-seed statistics.
2. A mechanism that predicts when hit rate misleads, in terms of the break-even length N^* and the superlinearity of prefill, and a closed-form decision rule that follows from it and requires no predictor: price each tier against N^*, and admit only what clears it.
3. The demonstration that hit rate and delivered value *reverse* on realistic workloads, and that this survives correcting the provisioning error the mechanism itself exposes: the hit-rate winner remains the value loser at 1.15× in a correctly provisioned two-tier hierarchy.
4. The finding that provisioning dominates policy: removing a tier whose restore cannot be repaid is worth 3.6× at identical total capacity, more than any eviction policy we evaluated.
5. Evidence that LRU is a stronger hit-rate baseline than the learned-caching literature assumes: it wins that metric on every constrained workload we test, with

all paired deltas significant at 95%.

6. An analytic break-even model for when restoration beats recomputation, in which N^* varies by four orders of magnitude with model architecture, validated against measured GPU and CPU prefill latencies.
7. Two self-reported evaluation defects, moving the verdict in opposite directions, and the observation that harness bugs flatter whichever side of a comparison depends on the broken component rather than perturbing rankings at random.

2 Background

2.1 Self-Attention and the KV Cache

Each transformer layer projects its input into query, key, and value matrices and computes scaled dot-product attention over all positions [18]:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \qquad (1)$$

During autoregressive decoding every new token attends to the keys and values of all tokens before it, so inference engines cache the per-layer K and V tensors rather than recompute them. Prefilling an N-token prompt costs $O(N^2)$ attention FLOPs, while the resulting cache grows linearly:

$$S(N) = 2 \cdot L \cdot H_{kv} \cdot d \cdot b \cdot N, \qquad (2)$$

for L layers, H_{kv} KV heads, head dimension d, and b bytes per element. Instantiated for the models in this paper (FP16): TinyLlama-1.1B (L=22, H_{kv}=4, d=64) holds 22 KB per token; Llama-2-7B $(32, 32, 128)$ holds 512 KB per token, or 2.1 GB for a single session at its full 4k context; Llama-2-70B's grouped-query attention $(80, 8, 128)$ holds 320 KB per token despite 10× the parameters. The footprint is material: when serving a 13B model on a 40 GB A100, roughly 65% of memory holds weights and over 30% holds KV cache [7]. VRAM is scarce, so eviction is unavoidable when multiple sessions compete.

2.2 Inference Is Memory-Bound

Compute has outpaced memory on modern accelerators: an A100 delivers 312 TFLOP/s of FP16 compute against 1.5–2.0 TB/s of HBM bandwidth, and both attention and the decode phase as a whole are bottlenecked by memory traffic rather than FLOPs [4, 7]. FlashAttention obtains its speedups purely by reducing bytes moved between HBM and on-chip SRAM, without changing the computation. Tiered KV persistence applies the same IO-centric accounting one level further down the hierarchy: when the KV bytes of idle sessions contend for a fixed-size fast tier, the governing questions are which bytes are worth their residency and what it costs to move them back, and both are questions about data movement, not compute.

2.3 Bélády's Algorithm

The provably optimal offline eviction policy [2] evicts the item whose next access occurs furthest in the future: $\arg\max_i \text{next_access}(i)$. For uniform-size, uniform-cost items, LRU is an effective online proxy by assuming temporal locality.

2.4 Weighted Caching

When items have varying sizes and costs, the uniform assumption fails. The problem maps to a dynamic Fractional Knapsack. As formalized by Young [20] and made practical by GreedyDual-Size [3], cost-aware eviction must account for value density per byte. LHD [1] further showed that for *dynamic* caches the correct denominator is not size alone but size × expected lifetime, which they call "hit density." In KV caching, "value" is the quadratic GPU compute saved by avoiding a cold-start prefill, and "weight" is the linear VRAM consumed; we evaluate both the static and lifetime-normalized forms in §9, and find the distinction matters less than the literature suggests once recompute costs are priced from the served architecture.

3 Related Work

LLM serving systems. PagedAttention [7] partitions the KV cache into fixed-size blocks, eliminating fragmentation and enabling memory sharing. vLLM manages VRAM dynamically but relies on reactive LRU eviction. CachedAttention [5] persists KV caches across multi-turn conversation rounds through a hierarchical cache, though its focus is architectural integration rather than learned eviction policies.

Attention state reuse. Prompt Cache [6] precomputes attention states for schema-defined prompt segments and splices them into new prompts, cutting TTFT by up to 8× on GPU and 60× on CPU. LMCache [11] and CacheBlend [19] focus on cross-request prefix sharing: caching system prompts shared across users. All three reuse state across prompts that share text; our work targets single-session persistence, saving user-specific conversational state over long time horizons, where the challenge is predicting which sessions will return.

Tiering and offloading. Mooncake [13] proposes a disaggregated KV cache architecture using a tiered hierarchy across nodes. FlexGen [14] offloads weights, activations, and KV cache across GPU, CPU, and disk to

maximize batch throughput on a single GPU, but its offload schedule is solved once for a static batch. Recent work persists per-agent quantized KV caches to disk for multi-agent inference on memory-constrained edge devices [15], reporting up to 136× TTFT reductions on restore. These systems engineer the persistence and placement mechanism; we target the complementary policy layer. Where they ask "how do we move and store cache state?", we ask "which entries should we keep when capacity is constrained, and by what metric should that choice be judged?"

Token-level KV compression. H2O [21] and Scissorhands [10] exploit the observation that a small set of pivotal tokens dominates attention scores to drop most of a sequence's KV entries (up to 5× reduction), and InfiniGen [8] offloads token-level state dynamically. CacheGen [9] instead compresses whole KV caches into quantized bitstreams for transmission, shrinking them 3.5–4.3× with negligible quality loss. Both lines are orthogonal to our granularity: we decide which *sessions* to retain across tiers, and token-level compression or KV-specific codecs could be composed with our policies to shrink what we store.

Learned cache eviction. LRB [16] learns to approximate Bélády's oracle for CDN caching; LHD [1] maximizes hits per unit of space-time occupied. Our work differs in the value model: CDN objects have roughly uniform miss costs, whereas KV cache entries have *quadratically* varying recompute costs. That changes what a hit is worth (§7.1) and, we argue, which metric should be reported.

4 System Design

Our architecture introduces a `TieredCacheManager` orchestrating three storage tiers:

1. **Hot Tier (VRAM):** In-memory storage. ∼0.1ms latency, 50MB capacity.
2. **Warm Tier (NVMe SSD):** Filesystem-backed with memory-mapped reads for files >10MB. ∼10ms latency, 150MB capacity.
3. **Cold Tier (S3/MinIO):** Object storage with Zstandard compression. ∼100ms latency, 300MB capacity.

The tier parameters mirror the hardware each stands in for: the hot tier models VRAM residency (sub-millisecond access), the warm tier a local NVMe device (∼10ms, 2 GB/s), and the cold tier compressed object storage (∼100ms, 500 MB/s). Capacities are scaled to the downscaled KV geometry of §8 so that the session-count-to-capacity ratio matches a production deployment.

Save path. `save` serializes a session's KV tensors and writes them to the hot tier. If the hot tier is full, the manager makes space by *cascading demotion*: the eviction policy selects a victim among hot entries, and that victim is demoted to the warm tier rather than deleted; if the warm tier is full, it recursively demotes warm victims to cold. Only a victim selected in the cold tier leaves the system. Eviction is therefore graceful by construction. A policy mistake at the hot or warm boundary costs a returning session only added restore latency; a mistake at the cold boundary destroys the session and forces a full re-prefill. This asymmetry is why victim selection, not tier engineering, is where policy quality matters.

Table 1: Tier transition pipeline

Transition	Serialization	Compression	Rationale
Hot → Warm	Raw Binary	None	Min. latency
Warm → Cold	Raw Binary	None (opt.)	§4
Cold → Warm	Raw Binary	—	Async prefetch

Load path. `load` consults an in-memory index and reads the entry from whichever tier holds it. A hit is recorded against that tier, and the value accounting of §8 discounts it by that tier's restore cost, so a cold-tier hit is worth less than a hot-tier one. On a warm or cold hit the entry is promoted back to the hot tier. A miss means the session either was never cached or fell out of the cold tier, and the serving layer must re-prefill the full history. `demote` and `promote` are also exposed directly for explicit migration.

Serialization uses a custom binary format with a 64-byte header (magic bytes, version, layer count, tensor dimensions, dtype) followed by packed FP16 tensor data. This is 3–5× faster than `safetensors` for hot↔warm transfers. All tier operations emit Prometheus metrics (hit rates by tier, migration latency distributions, per-policy value), and the repository includes a reference Grafana dashboard.

Is compression worth it? A natural cold-tier design is to compress caches before they reach object storage. Measurement says no, at least for general-purpose codecs: on a real serialized TinyLlama cache (Table 2), the best ratio any of LZ4, Zstandard, or zlib achieves is 1.10×. FP16 KV tensors are entropy-dense to byte-oriented codecs; the mantissa bits are effectively noise. The latency arithmetic is equally one-sided: at the cold tier's modeled 500 MB/s, restoring the 39 MB cache takes 80.7ms raw but 123.3ms through Zstandard and 93.8ms through LZ4, so compression costs latency while buying at most 10% capacity. Our evaluated configuration therefore stores raw FP16 at every tier (Zstandard remains available as an option). Meaningful KV compression requires format-aware codecs: CacheGen's quantization-and-bitstream encoding reaches 3.5–4.3× [9] precisely by not treating the tensors as opaque bytes, and composing such a codec with our cold tier is future work.

Table 2: General-purpose compression on a real serialized TinyLlama KV cache (1,792 tokens, 39.4 MB FP16; median of 5 trials, CPU). Restore = transfer at the cold tier's 500 MB/s + decompression; the uncompressed transfer takes 80.7ms. No codec pays for itself.

Codec	Ratio	Compress	Decompress	Restore
LZ4-0	1.01×	23ms	15ms	94.6ms
LZ4-9	1.01×	848ms	14ms	93.8ms
Zstd-3	1.10×	60ms	52ms	125.0ms
Zstd-19	1.10×	8,893ms	50ms	123.3ms
zlib-6	1.09×	1,290ms	140ms	213.8ms

5 Policy V1: Eviction as Classification

Our initial approach formulated eviction as supervised ML: predict $P(\text{resume})$ that a session will be accessed again within 1 hour. We engineered 9 features spanning the session's own state (age, token count, revisit count, minutes since last access), the clock (hour of day, day of week, a business-hours indicator), and the user's history (historical return rate, mean session length), and trained Logistic Regression and Gradient Boosted Trees on 7-day persona-driven traces (80/20 split, standardized features, class-balanced loss; we select by ROC-AUC rather than accuracy, which label imbalance inflates). Test AUC is 0.70 (logistic) and 0.73 (GBT); we use the logistic model in the main study for interpretability. Its strongest positive coefficient is the user's historical return rate (+0.75: one standard deviation multiplies the odds of resumption by $e^{0.75} \approx 2.1\times$), which confirms the personas carry learnable signal. Candidate sets are scored in a single batched prediction per eviction sweep.

Negative Result: Under enterprise load, Logistic V1 achieves a 67.9% hit rate [66.8, 69.0] versus LRU's 79.1% [75.9, 82.4]. The hypothesis is falsified: an 11.2-point deficit, reproduced on every workload, that no amount of feature engineering recovered. We took this at face value for most of the study. Only after correcting the value model (§9.1) did we find that the same policy saves 1.73× more GPU time than the baseline it loses to (Table 3), which is the observation the rest of this paper is about.

6 Failure Analysis: Eviction Is a Packing Problem

This exposed a flaw in the formulation itself: eviction under capacity constraints is not a classification problem; it is a knapsack variant. LRU accidentally approximated correct behavior by rotating entries uniformly. Our ML models confidently kept large "high probability" sessions that choked capacity, forcing out dozens of smaller entries.

The arithmetic of the failure is worth stating concretely. Under the enterprise workload, an 8k-token session occupies the space of sixteen 512-token sessions. V1 retains it whenever its resumption probability exceeds each smaller entry's individually, say 0.9 against 0.6. But the sixteen displaced sessions carried $16 \times 0.6 = 9.6$ expected future hits against the large entry's 0.9. The classifier answers the question "will this session return?", while the capacity constraint asks a different one: "what is the most valuable *set* of sessions that fits?" The binary criterion $\min P(\text{resume})$ ignores both entry size and variable recompute cost. The oracle ablation of §9.6 later sharpens this diagnosis: with ground-truth $P(\text{resume})$, this same objective achieves a perfect cache at our capacity, so the formulation is not intrinsically broken. Its flaw is how it behaves under prediction error: a confident false positive on one large session forces out dozens of smaller entries, so the objective amplifies the predictor's mistakes in proportion to entry size. The reformulations that follow change what is being maximized so that errors cost less.

7 Policies V2 and V3: Eviction as Packing

We reformulated the objective based on weighted caching theory:

$$\text{Value Density} = \frac{P(\text{resume}) \times \text{RecomputeCost}(N)}{\text{SizeBytes}(N)} \quad (3)$$

Because prefill scales $O(N^2)$ but memory scales $O(N)$, value density grows linearly with context length: doubling a session's tokens doubles the recompute saved per byte retained. Under this objective the policy should, at any instant, pack the cache with the highest-density entries, which is precisely the greedy solution to the fractional knapsack. We also implemented an **Admission Control Gate**: sessions below a value-density threshold are denied entry to the warm tier, on the theory that filtering low-value entries at the door preserves capacity for high-value ones.

7.1 Space-Time Density: A Cache Is a Dynamic System

Little's Law suggests a failure mode for static value density. We state the resulting objective here because it is one of the policies we evaluate, and we state its outcome up front: the failure mode is real but narrow, and lifetime normalization earns its keep only under high size variance (§9.2). An earlier version of this paper reported it

as general, on the strength of a cost model we have since corrected. The reasoning is that a cache is not a static knapsack but a dynamic system, in which each entry occupies *space over time*, so the objective must account for expected sojourn time $\mathbb{E}[\Delta t]$:

$$\text{Space-Time Density} = \frac{P(\text{resume}) \times \text{RecomputeCost}(N)}{\text{Size}(N) \times \mathbb{E}[\Delta t]} \tag{4}$$

Under Little's Law, $L = \lambda \times W$, where L is the steady-state number of resident entries, λ their admission rate, and W their average sojourn time; the law holds for any stationary arrival process, independent of distributional assumptions. At full utilization the byte capacity is fixed, so L equals capacity divided by mean entry size. A static density score never decreases as an entry idles (its numerator and denominator are both time-free), so the highest-density entries, which are the largest, are simply never evicted: their W grows without bound, mean entry size rises, and L collapses. Fewer resident sessions means fewer opportunities for any future request to hit. On this reasoning the policy would win every local comparison of value per byte while destroying the aggregate through the quantity it never measures: time.

This objective is structurally an adaptation of LHD's hit density [1] (expected hits per unit of size $\times$ lifetime) to the KV-cache setting. The adaptation is non-trivial in two respects: the numerator weights each hit by a *quadratic* recompute cost rather than counting hits uniformly, and $P(\text{resume})$ comes from a learned session-level model rather than age-binned statistics.

V3 Implementation. We estimate $\mathbb{E}[\Delta t]$ per session as an exponential moving average (α=0.3) of observed inter-access gaps, with a 2-hour prior for sessions with no observed gap. Because an entry idle *longer* than its typical rhythm should expect a longer remaining wait, not a shorter one (the inspection paradox), we score with $\max(\text{EMA gap}, \text{current idle time})$. Candidate sets are re-scored on each eviction sweep.

8 Experimental Setup

Policies under test. Six policies compete on identical traces. *LRU* evicts the least-recently-accessed entry. A *recency–frequency heuristic* scores each entry $0.4f + 0.4r + 0.2v$, where f is its access count normalized over the candidate set, $r = \exp(-\ln 2 \cdot \Delta t / 30\text{min})$ decays with idle time Δt, and v is its normalized token count; the lowest score is evicted. *Logistic V1* evicts by lowest predicted $P(\text{resume})$ (§5). *Value Density V2* and its admission-controlled variant *V2+AC* evict by lowest value per byte, and *Space-Time V3* by lowest lifetime-normalized value density (§7.1). LRU and the heuristic consume only access order and counts; the remaining policies consume session features or learned predictions, which is exactly the dependence that §9.1 shows makes them fragile.

Workloads. We evaluated eviction policies using a synthetic workload simulator with three profiles via Poisson arrivals and Log-normal token distributions:

- **Casual:** 50/hr arrival, μ=512, σ=0.8, low return probability.
- **Enterprise:** 200/hr, μ=2048, σ=0.5, high return probability.
- **Power User:** 80/hr, μ=8192, σ=1.2, medium return probability.

User Personas. Each simulated user carries a persistent persona (a return-propensity multiplier, a verbosity scale, and a 10-hour diurnal activity window) drawn once and reused across all of that user's sessions. Without personas, per-user features such as historical return rate are i.i.d. noise by construction and no predictor can exceed chance (we measured AUC 0.56 under an i.i.d. pool vs. 0.70 with personas). Conversations grow monotonically: each resume appends new turns to the previously cached context.

Virtual Clock. Traces are replayed on a simulated clock advanced to each event's timestamp. All components (the cache manager, eviction policies, and feature extraction) read time exclusively from this clock. This is essential for validity: replaying a 6-hour trace at wall-clock speed collapses every temporal feature (session age, idle time, hour of day) to near zero. That silently invalidates any policy that reasons about time, while order-based policies like LRU are unaffected.

Model Configuration. KV cache sizes are computed via Eq. 2 with a downscaled configuration (2 layers, 2 heads, head dimension 32, FP16; 512 bytes/token). Llama-2-7B geometry (§2.1) is exactly 1,024$\times$ larger per token; equivalently, our 500 MB capacity stresses the same session-count-to-capacity ratio as a $\sim$500 GB tier would for 7B-scale caches. Scaling all entries by a constant shifts absolute capacity dynamics but not relative policy comparisons.

Simulation. 6-hour continuous window; token counts clamped to 8,192 to preserve heavy-tailed distributions.

Capacity. 500MB total (50MB hot + 150MB warm + 300MB cold) in the main study; §9.7 sweeps 125MB–2GB at the same 10/30/60 split.

GPU Savings. Each hit is credited with the prefill it avoids for the tokens *actually cached* (not the grown context at resume time), minus the modeled cost of restoring the entry from the tier where it was found. Both sides are priced from one architecture using the same expressions as the break-even model of §11: prefill costs $(2PN + 4LH_q dN^2)/(F\eta)$ and restore moves $2LH_{kv}dbN$ bytes at the tier's latency and bandwidth. Pricing both sides together is what makes the accounting coherent: a

hit whose restore exceeds the prefill it avoids is credited zero rather than counted as a win, so the value model agrees with §11 about which hits are worth having.

We report value as **GPU-seconds saved per day**. Dollars are a linear rescaling ($2.50/hr for an on-demand A100 80GB) that we quote where it aids interpretation, but the GPU-second is the scale-free quantity: it does not depend on the reader's negotiated GPU price, and at our workload's modest arrival rates the dollar figures say more about deployment size than about policies.

On the size of these numbers. Our enterprise workload issues 200 sessions per hour, so its absolute figures are small: LRU's 36.8 GPU-s/day is about three cents. That is a statement about the simulated deployment, not about the effect, and we normalize two ways so the reader need not do this arithmetic themselves. Per unit of traffic, the workload serves roughly 6,300 cache loads per day, so LRU delivers 5.9 GPU-seconds per thousand loads against value density's 11.7, an advantage of 5.8 GPU-seconds per thousand. We report the per-thousand-loads figure rather than extrapolating to a production arrival rate, because such an extrapolation would assume cache dynamics unchanged across five orders of magnitude of load, which we have not simulated.

Architecture is an experimental variable, not a constant. Whether a hit is worth anything depends on the model being served, because prefill scales superlinearly in N while restore scales linearly. We therefore run the matrix under two architectures: **TinyLlama-1.1B** (22 KB/token; the model our end-to-end validation measures, §10) and **Llama-2-70B** (320 KB/token, GQA; representative of models large enough that persistence is deployed in practice). We also report Llama-2-7B as a limiting case: its MHA cache is so large per token that *no* session in our workload clears break-even, making persistence the wrong design there rather than a policy problem (§11).

Statistics. Every (policy, workload) cell is run over 10 independent seeds. All policies replay the identical trace per seed, so policy comparisons are *paired*: we report per-seed deltas against LRU with 95% t-based confidence intervals across seeds.

8.1 Real-Trace Workload: Azure LLM Inference

To test external validity beyond synthetic workloads, we additionally replay the Azure LLM inference trace [12, 17]: 27.3M production conversation requests spanning one week of May 2024, each carrying an arrival timestamp and context/generated token counts. Two properties of the trace transfer into our replay verbatim: the *arrival process* (real burstiness and diurnal/weekly load shape; timestamps keep their true UTC hour and weekday on the simulator's Monday-00:00 time axis) and *per-request context sizes* (heavy-tailed: median 928, mean 1,632, p95 4,943 tokens).

What the trace cannot provide is session identity: for customer privacy it carries no conversation identifiers, and we find linkage is not reconstructable either. A greedy matcher that chains requests by context-token growth (a follow-up turn's context must contain the previous turn's context plus its response) links 87.8% of requests, but it links an indistinguishable 87.8% on a *permutation control* in which token counts are shuffled across requests, which destroys any true chains. At production density ($\sim$60 req/s over at most 8,000 distinct context lengths) growth-chaining is collision-dominated, a caution for any study tempted to reconstruct conversations from these public traces. We therefore impose return behavior using the same persona machinery as the synthetic workloads (enterprise return-window parameters), with resume-time context growth drawn empirically from the window's generated-token distribution. Differences between the synthetic-enterprise and real-trace results are thus attributable to the arrival process and size distribution, which are real; return behavior is held fixed and modeled.

We slice ten 6-hour windows across the week (covering weekday/weekend and peak/trough load) and thin each to 300 session starts per hour by taking every k-th request, preserving the arrival shape. The thinning rate is chosen for capacity pressure: real Azure requests are markedly smaller than synthetic enterprise sessions, so at the enterprise arrival rate (200/h) the 500 MB cache is unconstrained (LRU $\approx$98% hit rate, no policy differentiation, as with the casual workload), while 300/h restores sustained eviction pressure comparable to the enterprise regime. Each window is one replicate: all six policies replay the identical event list, so deltas remain paired, with windows in the role of seeds. The learned policies reuse the predictor trained on synthetic personas, a zero-shot transfer test. The conversion is implemented in `benchmarks/azure_trace_loader.py`.

9 Comparative Evaluation

Our evaluation is organized around seven questions. Does learned or value-aware eviction beat LRU under an artifact-free protocol? (§9.1: it wins hit rate everywhere, and loses value on enterprise.) Does the Little's Law diagnosis of §7.1 hold, i.e. does lifetime normalization improve on static value density? (§9.2: only under high size variance; the collapse we had previously reported was a cost-model artifact.) Do hit rate and value actually diverge enough to mislead? (§9.3: rankings invert.) When the policies are handed the trace's actual future, is the bottle-

Table 3: Policy comparison under TinyLlama-1.1B economics (500MB, 6hr, 10 seeds; Δ is the *paired* per-seed delta vs. LRU in GPU-seconds saved per day, with 95% t-CI). LRU wins hit rate on both workloads and loses value on enterprise to every value-aware policy. Casual omitted: unconstrained, all policies at 100%.

Policy	Workload	Hit%	GPU-s/day	Δ vs LRU [CI]
LRU	Enterprise	**79.1**	36.8	—
Heuristic	Enterprise	55.3	60.7	+23.9 [+20.8, +27.0]
Logistic V1	Enterprise	67.9	63.5	+26.7 [+23.1, +30.3]
ValDensity V2	Enterprise	58.2	**73.2**	+36.4 [+30.4, +42.4]
ValDens+AC	Enterprise	58.2	68.4	+31.6 [+25.5, +37.7]
SpaceTime V3	Enterprise	57.8	72.4	+35.6 [+31.5, +39.6]
LRU	Power User	**96.0**	48.2	—
Heuristic	Power User	84.2	**51.9**	+3.7 [+0.7, +6.8]
Logistic V1	Power User	83.9	46.2	−2.0 [−5.1, +1.1]
ValDensity V2	Power User	76.9	46.5	−1.7 [−5.0, +1.6]
SpaceTime V3	Power User	79.6	50.7	+2.5 [−1.6, +6.6]

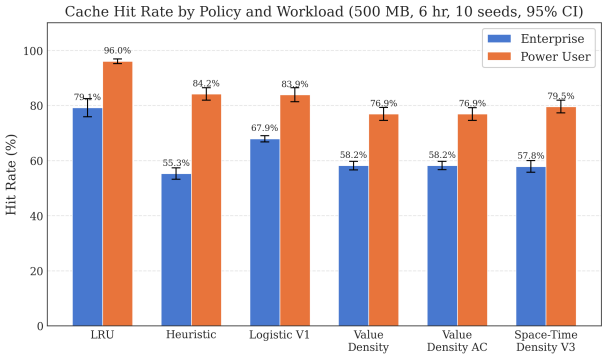


Figure 1: Cache hit rate by policy and workload, 95% CIs across 10 seeds. LRU wins this metric on both constrained workloads; Figure 2 shows what that does *not* imply.

neck the predictor or the objective? (§9.6: the predictor; every oracle achieves a perfect cache.) How far do these verdicts travel across capacities? (§9.7: the advantage is monotone in scarcity, from 7.97× at 125 MB to 1.05× at 2 GB, and never reverses.) Does any of this survive contact with real production traffic? (§9.8: every finding replicates and the inversion strengthens.) And how much of it depends on our having generated the resumption signal ourselves? (§9.9: the learned policy's standing does; the metric finding does not.)

We evaluated 6 policies across 3 synthetic workloads over 10 seeds, plus the real-trace replay of §8.1. The casual workload achieved 100% hit rate at 500MB (the cache was never constrained), so Table 3 focuses on the two workloads where eviction pressure forces policy differentiation.

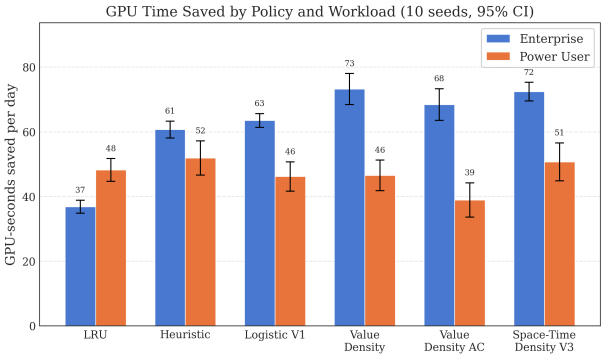


Figure 2: GPU time saved per day, 95% CIs, same runs as Figure 1. On enterprise the ordering *reverses*: LRU, the hit-rate winner, saves the least. On power-user, where sessions sit far above the break-even length, the two metrics largely agree.

9.1 LRU Is a Stronger Baseline Than Assumed

LRU wins the metric we set out to beat. Under virtual-clock replay with paired multi-seed statistics, it attains the highest hit rate on every constrained workload: 79.1% [75.9, 82.4] on enterprise against Logistic V1's 67.9% [66.8, 69.0], and 96.0% against 83.9% on power-user. No learned or value-aware policy we built comes close. Our original hypothesis is falsified as decisively as the data permits, and we recommend LRU without reservation to anyone whose objective really is hit rate.

That recommendation is narrower than it sounds, because on the enterprise workload LRU simultaneously delivers the *least* value of any policy in Table 3 (§9.3). The two statements are consistent, and their combination is this paper's subject.

Two bugs, both flattering, both ours. We arrived here by correcting our own pipeline twice, and the corrections moved the verdict in opposite directions. First, an early harness replayed traces at wall-clock speed and credited hits with unclamped resume-time token counts; it showed a learned policy beating LRU by $68/day. That was an artifact: collapsing simulated time zeroed the temporal features the model was trained on, and the accounting credited a different quantity than the policies optimized. Fixing it reversed the result, and we long reported LRU as winning on both metrics.

Second, and only after the analysis in this paper was otherwise complete, we found that our cost model contained no architecture at all: it priced a prefill with two hardcoded constants whose linear/quadratic balance placed the crossover at ∼1,300 tokens, where real architectures cross between 12k and 53k (§8). That is not a rescaling, and the invariance argument we had relied on,

that a constant factor cannot change rankings, did not apply: the error inflated an 8k-token session's worth relative to a 512-token one by 3–4.5×, exaggerating precisely the effect the study measures. Re-pricing from architecture (and validating the new model against measured hardware, §10) inverted the value comparison while leaving hit rates untouched.

The pattern is worth naming, because it is not symmetric. A policy that consumes only access order is insensitive to broken time, broken features, *and* broken value models; every richer policy is exposed to all three. Bugs in an evaluation harness therefore do not perturb rankings randomly. They flatter whichever side of the comparison depends on the broken component: our time bug flattered the learned policy, our cost-model bug flattered LRU. We report both, and note that we caught the second only when a review of the cost model compared its 57.7-second prediction for a 598-token prefill with the ∼37ms the hardware measured.

9.2 Lifetime Normalization Helps Only Under High Size Variance

Value density trades cardinality for value deliberately: V2 holds 58.2% hit rate against LRU's 79.1% on enterprise while delivering the most value of any policy we tested. Its retained set is smaller and worth more, which is the intended behavior of a packing objective and, under correct economics, the winning one.

The space-time correction (V3) divides that same density by $\mathbb{E}[\Delta t]$ to bound sojourn time. It earns its keep only where sessions vary widely in size: on power-user (σ=1.2) V3 beats V2 by +4.2 GPU-s/day (paired 95% CI $[+2.3, +6.1]$) with +2.6 hit-rate points, while on enterprise (σ=0.5) the two are statistically indistinguishable ($-0.8\,[-3.6, +1.9]$). This is what the Little's Law argument of §7.1 should predict on reflection: unbounded sojourn hurts when a policy can hoard entries far larger than average, and heavy-tailed size distributions are what create that opportunity. Admission control (V2+AC) never helps and costs 4.8–7.6 GPU-s/day, confirming that under sustained load the cache runs at near-full utilization regardless of filtering at the door.

A retraction. An earlier version of this paper reported that static value density *collapsed*, worst hit rate of any policy, with V3 recovering $94/day over it, and offered that as evidence for the Little's Law diagnosis. That result did not survive the cost-model correction of §9.1. The superseded model overstated a large session's worth relative to a small one by 3–4.5×, which is precisely the signal V2 maximizes; V2 hoarded accordingly and starved its own cardinality. Priced correctly, the same policy hoards less and wins. This is the clearest instance of a pattern we return to in §9.1: the collapse we had diagnosed as a property of the *objective* was a property of our *cost model*, and only a value-aware policy could have been misled by it.

9.3 Hit Rate and Value Decouple

On the enterprise workload the two metrics do not merely diverge, they *reverse*. LRU leads on hit rate by 11.2 points over Logistic V1 and by 20.9 over Value Density V2, yet it delivers the least value in the table: V2 saves 1.99× as much GPU time (paired +36.4 GPU-s/day $[+30.4, +42.4]$), V3 1.97×, V1 1.73×, and even the crude recency–frequency heuristic 1.65×. Every one of those deltas is significant. Tuning for hit rate on this workload selects the lowest-value policy in the table.

Why the reversal happens. Hit rate weights every hit identically; value does not, for two compounding reasons visible in our accounting. First, a *threshold* effect: a hit is worth nothing at all unless the prefill it avoids exceeds the restore it costs, i.e. unless the session exceeds the break-even length N^* of §11. Under TinyLlama economics N^* is 1,561 tokens from NVMe and 25,901 from object storage, so *every* cold-tier hit in our token range is worth exactly zero. LRU, which is blind to both size and tier, earns 812 of its 1,237 enterprise hits in the cold tier; they inflate its hit rate and contribute nothing. Second, a *magnitude* effect: above N^*, savings grow superlinearly in N, so retaining one 8k-token session can outweigh a dozen 2k-token ones. Value-aware policies concede many cheap hits to keep a few expensive ones, which is exactly the trade hit rate cannot express.

This also predicts when the metrics agree, which we confirm in two independent ways. On *power-user* (median 8,192 tokens, far above N^*), nearly every hit clears the threshold and the metrics reconverge: no policy beats LRU by more than 1.08× and most differences are not significant (Table 3). And on *Llama-2-70B* economics, where grouped-query attention puts N^* at 12 tokens from NVMe so that essentially all hits are valuable, LRU's hit-rate advantage largely does translate into value (§9.4). The metrics coincide when cached items are near-uniform in value and diverge as that uniformity breaks. Uniformity of value is the assumption hit rate encodes, and heavy-tailed LLM workloads violate it.

The right target for KV-cache eviction, in both optimization and evaluation, is expected cache value, not hit rate.

9.4 The Served Architecture Sets the Regime

Re-running the identical traces priced at Llama-2-70B separates the two effects, because grouped-query attention moves N^* from 1,561 tokens to 12 and thereby switches

Table 4: The same traces priced at Llama-2-70B economics (320 KB/token, N^*=12 from NVMe, so every hit clears break-even). Compare Table 3: removing the threshold effect shrinks the enterprise gap from 1.99× to 1.07× and hands power-user decisively back to LRU.

Policy	Workload	Hit%	GPU-s/day	Δ vs LRU [CI]
LRU	Enterprise	**79.1**	8,173	—
Heuristic	Enterprise	55.3	7,556	−618 [−796, −439]
Logistic V1	Enterprise	67.9	**8,806**	+633 [+266, +1000]
ValDensity V2	Enterprise	67.1	8,722	+549 [+227, +870]
SpaceTime V3	Enterprise	63.2	8,783	+610 [+384, +835]
LRU	Power User	**96.0**	**4,576**	—
Heuristic	Power User	84.2	4,366	−209 [−303, −115]
Logistic V1	Power User	83.9	3,958	−618 [−744, −491]
ValDensity V2	Power User	82.7	3,961	−615 [−747, −483]
SpaceTime V3	Power User	84.7	4,119	−456 [−579, −334]

the threshold effect off while leaving the magnitude effect intact.

Table 4 confirms the decomposition. On *power-user*, where sessions were already far above TinyLlama's N^*, eliminating the threshold hands LRU an unambiguous win: it leads every policy by 0.87–0.95×, all significant, precisely the outcome hit rate would have predicted. On *enterprise*, a residual inversion survives, V1, V2, and V3 still beat LRU while conceding 11–16 hit-rate points, but the margin collapses from 1.99× to 1.07–1.08×. Splitting the effects this way, the threshold accounts for roughly a 1.85× advantage and the magnitude effect for the remaining ∼1.08×.

The magnitude effect is small at 70B for a reason the cost model makes explicit: prefill's quadratic attention term does not overtake its linear weight term until N≈53,400 tokens for this architecture, so within our context range a 70B prefill is nearly *linear* in N. Value is then close to proportional to total tokens served from cache, which is very nearly what hit rate measures when sizes are comparable. Hit rate is therefore most reliable for large models and least reliable for small and mid-sized models served from slow tiers. This is the opposite of where intuition anchored on absolute serving cost would place the risk.

9.5 Provisioning Dominates Policy

Section 9.3 attributes most of the enterprise inversion to the threshold effect, and under TinyLlama economics that effect has a stark cause: N^* from object storage is 25,901 tokens against a workload clamped at 8,192, so a cold-tier hit can never repay its restore. A skeptical reading follows. We built a three-tier hierarchy, evaluated it on an architecture where the third tier cannot pay, and found that policies avoiding that tier win. On that reading our headline measures provisioning, not eviction.

Table 5: Two-tier control (enterprise, 10 paired seeds, TinyLlama economics, 500 MB in both configurations). Removing a tier that cannot repay its restore is worth more than any policy choice within the hierarchy; the metric inversion survives the removal at reduced magnitude.

	Three-tier		Two-tier	
Policy	Hit%	GPU-s/d	Hit%	GPU-s/d
LRU	79.1	36.8	79.4	**131.0**
Heuristic	55.3	60.7	56.5	133.4
Logistic V1	67.9	63.5	68.0	**150.7**
ValDensity V2	58.2	**73.2**	58.5	147.4
SpaceTime V3	57.8	72.4	58.1	150.0
Best ×LRU		1.99×		1.15×

We test it directly. The control removes the cold tier and redistributes its capacity to hot and warm, holding total bytes fixed at 500 MB, and replays the identical traces.

Three results follow, and the first is the largest effect in this paper.

Provisioning dominates policy. Deleting the cold tier raises LRU from 36.8 to 131.0 GPU-s/day, a factor of 3.6, at identical total capacity. No eviction policy achieves anything comparable within the fixed hierarchy, where the best margin is 1.99×. Two-tier LRU, the simplest policy in the simplest hierarchy, delivers 1.8× more value than the best three-tier policy we built. The same bytes provisioned differently are worth more than the same bytes managed better, and the rule that decides this is arithmetic: compare N^* for each tier against the workload's context-length distribution, and do not build a tier whose restore cannot be repaid.

The metric inversion is not an artifact of that provisioning error. With the cold tier gone and every hit served from a tier that can pay, LRU still wins hit rate (79.4%) and still loses value: Logistic V1 and Space-Time V3 each deliver 1.15× more (+19.7 GPU-s/day [+14.8, +24.6] and +19.0 [+13.1, +25.0]), and Value Density V2 1.13×, all significant. The reversal is smaller than the 1.99× we report in Table 3 but it does not vanish, so it does not reduce to "value-aware policies avoid a useless tier."

The decomposition is confirmed independently. Section 9.4 estimated the threshold and magnitude components at ≈1.85× and ≈1.08× by switching architectures. This control estimates the magnitude component at 1.13–1.15× by switching hierarchies instead, holding the architecture fixed. Two different manipulations, each leaving roughly 1.1×, isolate the same residual effect: above N^*, savings grow superlinearly, so retaining fewer larger sessions beats retaining more smaller ones even when every hit is worth something. Hit rate cannot express that trade at any tier count.

We therefore state the headline of Table 3 with its con-

Table 6: Oracle ablation under corrected economics (500MB, 10 seeds, identical traces to Table 3). All three oracles cache perfectly; their hit rates are indistinguishable at 100.0%, yet their delivered value is not.

Policy	Workload	Hit%	GPU-s/day	Δ vs LRU [CI]
LRU	Enterprise	79.1	36.8	—
Bélády	Enterprise	100.0	209.8	+173.0 [+167.5, +178.6]
Oracle V1	Enterprise	100.0	209.8	+173.0 [+167.5, +178.6]
Oracle V3	Enterprise	100.0	**212.7**	+175.9 [+170.2, +181.7]
LRU	Power User	96.0	48.2	—
Bélády	Power User	100.0	**122.7**	+74.5 [+66.5, +82.5]
Oracle V1	Power User	100.0	122.7	+74.5 [+66.5, +82.5]
Oracle V3	Power User	100.0	122.1	+74.0 [+66.1, +81.8]

dition attached. The 1.99× figure holds for a three-tier deployment serving a small model, which is a configuration our own break-even model advises against building. The portable claim is the 1.15× that survives correct provisioning, together with the observation that finding the provisioning error at all required pricing hits by what they save rather than counting them.

9.6 Ground Truth: Prediction Is the Binding Constraint

The results so far leave an alternative explanation open: perhaps the learned policies lose not because their objectives are wrong but because their predictor is weak (test AUC 0.70). To separate the two, we replay the same traces with *oracle* policies that read the trace's actual future, which is legitimate here because replay is open-loop: arrivals never depend on cache behavior. Three oracles span the objective space. *Belady* evicts the entry whose next access is farthest away, the classical clairvoyant baseline. *Oracle-V1* runs V1's exact objective with a perfect classifier: P(resume within 1h) is the ground-truth indicator, with ties broken toward sooner-returning entries. *Oracle-V3* runs V3's objective with perfect inputs: the ground-truth return indicator and the exact time to next access in place of the EMA estimate, making it a value-weighted greedy Belady.

Table 6 yields three results. First, perfect prediction yields a perfect cache: every oracle attains a 100% hit rate on both constrained workloads, so at 500 MB the set of sessions that will ever return always fits. Every miss in Table 3 is a prediction failure, not a capacity necessity.

Second, the headroom above LRU is larger than its hit-rate deficit suggests. LRU trails the oracles by 20.9 hit-rate points on enterprise but by 5.70× in value (+173.0 GPU-s/day [+167.5, +178.6]), and by 4.0 points but 2.55× on power-user. The ratio is inflated relative to the point gap for the reason developed in §9.3: a clairvoyant policy not only hits more often, it hits on sessions that are worth something and serves them from tiers where the restore is cheap, whereas much of LRU's hit rate is spent on cold-tier entries worth zero.

Table 7: Capacity sweep (enterprise, TinyLlama economics, 5 paired seeds; the main matrix in Table 3 uses 10, which is why its 500 MB row reads 79.1%/36.8 against 80.3%/37.9 here). Value-aware eviction beats LRU at every capacity, by 7.97× when the cache is starved and still by 1.05× when it is large enough that *every policy hits 100%*. Hit rate saturates; value does not.

Cap. (MB)	LRU hit%	LRU GPU-s/d	V2 ×LRU	V3 ×LRU	V1 ×LRU	Bélády GPU-s/d
125	14.5	2.1	7.38	**7.97**	3.28	73.4
250	36.7	12.2	3.14	3.22	2.06	134.1
500	80.3	37.9	1.90	1.91	1.66	209.8
1000	99.0	99.6	1.25	1.32	1.30	281.7
2000	100.0	190.7	1.00	1.05	1.08	347.0

Third, hit rate cannot distinguish the two oracles, yet their delivered value differs significantly. Bélády and Oracle V3 both cache perfectly, 100.0% versus 100.0%, a paired difference of exactly zero on every seed, but Oracle V3 delivers +2.9 GPU-s/day more on enterprise $[+2.0, +3.8]$ and 0.5 less on power-user $[-0.8, -0.3]$, both significant. Once no session need ever be dropped, the only remaining decision is *which tier* holds each entry, and hit rate is by construction blind to it: Bélády orders entries by next access time, while Oracle V3 weights that ordering by recompute value and so keeps expensive sessions in fast tiers. Prediction error, capacity pressure, and policy heuristics are all absent here: two clairvoyant policies with identical hit rates deliver measurably different value.

These differences are small in absolute terms (1.4% and 0.4%). We do not claim tier placement is a large effect, only that it is a measurable one that hit rate reports as zero.

9.7 Capacity Changes the Size of the Gap, Not Its Direction

Any single-capacity study has implicitly chosen an operating point. We therefore sweep total capacity from 125 MB to 2 GB on the enterprise workload, holding the 10/30/60 tier split and using five paired seeds per point, and include Bélády as an upper reference. (We also ran Oracle V3 at every capacity; it tracks Bélády to within 2%, so we omit it here and report the comparison in §9.6.)

Three observations follow from Table 7. First, the value-aware advantage is *monotone in scarcity*: 7.97× at 125 MB, 1.91× at our canonical 500 MB, 1.05× at 2 GB. Squeezing the cache does not change which policy wins, only by how much, so our canonical operating point is

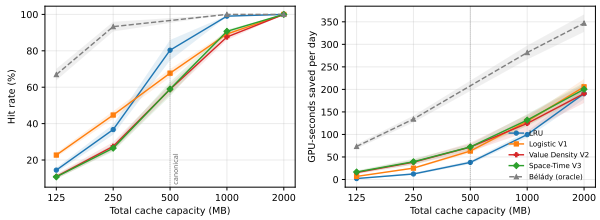


Figure 3: Capacity sweep (enterprise, 5 paired seeds, shaded 95% CIs). Left: hit rate converges to 100% for every policy as capacity grows. Right: delivered value does not converge. The dotted line marks the canonical 500 MB operating point.

Table 8: Azure LLM inference trace (§8.1): ten 6-hour windows as paired replicates, TinyLlama economics, 500MB. Every alternative beats LRU on value while losing to it on hit rate; all paired deltas significant.

Policy	Hit%	GPU-s/day	Δ vs LRU [CI]
LRU	**83.9**	32.1	—
Heuristic	60.9	53.2	+21.2 [+18.8, +23.5]
Logistic V1	73.8	59.1	+27.0 [+24.5, +29.5]
ValDensity V2	67.0	**70.0**	+37.9 [+34.8, +41.1]
ValDens+AC	66.8	63.9	+31.9 [+28.8, +34.9]
SpaceTime V3	68.8	62.7	+30.6 [+28.6, +32.6]

neither the best nor the worst case for the argument.

Second, the direction never reverses. At 125 MB, Space-Time V3 attains a *lower* hit rate than LRU (10.7% versus 14.5%) while delivering nearly eight times the value, because 83% of LRU's hits land in the cold tier where, under TinyLlama economics, a hit is worth exactly zero.

Third, the metric fails where it appears most reliable. At 2 GB every policy in the table reaches a 100.0% hit rate, so hit rate reports them as *exactly equivalent* and has no remaining power to discriminate at all. Their delivered value nonetheless spans 190.7 to 347.0 GPU-s/day, a factor of 1.82. The tier histograms explain it completely: Bélády serves 87% of its hits from the hot tier against LRU's 28%, because when nothing must ever be evicted the only decision left is placement, and a policy that ranks purely by recency lets valuable sessions drift into slower tiers. This is the same effect isolated in §9.6, here made large by a capacity regime that removes every other source of difference. A deployment provisioned generously enough to reach a perfect hit rate would conclude from that metric that its eviction policy no longer mattered, which Table 7 shows is false.

9.8 Findings Replicate on Real Traces

Replayed over real production arrivals and sizes, both halves of our finding replicate, and the second one strengthens. LRU again wins hit rate outright, by 10.1 points over Logistic V1 and 16.9 over Value Density V2, so the falsification of our original hypothesis holds on real arrival processes as well as synthetic ones. And LRU again delivers the least value of any policy tested: V2 saves 2.18× as much GPU time (paired +37.9 GPU-s/day [+34.8, +41.1]), V3 1.95×, and even the untrained recency–frequency heuristic 1.66×, every delta significant across the ten windows.

The inversion is *larger* here than on synthetic enterprise traffic (2.18× versus 1.99×), and the reason is visible in the trace statistics of §8.1: real request sizes are more heavily skewed (median 928 tokens, mean 1,632, p95 4,943) than our log-normal enterprise profile, so a larger fraction of sessions falls below the break-even length N^*=1,561 where a hit is worth nothing. The mechanism of §9.3 predicts exactly this: the more mass a workload places near and below N^*, the more hit rate overstates what a policy is delivering. That the learned policies achieve this while running zero-shot, on a workload whose arrival process they were never trained on, makes the result harder to attribute to overfitting.

9.9 How Much Does the Verdict Depend on Synthetic Predictability?

Our resumption signal is generated, not observed. Each simulated user carries a persona whose return propensity is drawn log-normally with dispersion σ, and that dispersion sets a ceiling on how much per-user structure any predictor can recover. The AUC of 0.70 reported in §5 is therefore a property of our generator as much as of our model, and a reader is entitled to ask whether the conclusions are an artifact of choosing σ favorably. We test this directly by sweeping σ, retraining the predictor at each setting so the model is as good as that world permits, and re-running the enterprise matrix.

Table 9 separates two claims that the main study confounds. The *learned policy's* standing does depend on σ, as it must: at σ=0, where users are behaviorally identical and per-user features carry no information, Logistic V1 falls to 0.90× LRU's value and the difference is not significant. A classifier whose premise is per-user prediction should fail when there is no per-user structure to predict, and it does.

The *metric* finding does not depend on σ. Space-Time V3 beats LRU on value at every setting, from 1.34× [+5.9, +14.9] GPU-s/day at σ=0 to 2.25× [+42.1, +60.0] at σ=1.0, and LRU leads on hit rate at every setting except the last. This is what the mecha-

Table 9: Persona strength sweep (enterprise, 5 paired seeds, TinyLlama economics). σ sets between-user dispersion in return propensity and therefore how much signal a predictor can learn; the model is retrained at each setting. Value is GPU-seconds saved per day. Every V3 delta is significant. Bold marks the highest hit rate in each row: LRU leads at every dispersion except the strongest, where Logistic V1 overtakes it by 2.0 points.

σ	AUC	LRU hit%	LRU GPU-s/d	V1 hit%	V1 ×LRU	V3 hit%	V3 ×LRU
0.0	0.659	**95.3**	30.3	49.8	0.90	49.7	1.34
0.3	0.670	**89.5**	33.8	71.9	1.47	66.6	1.50
0.6	0.700	**80.3**	37.9	67.7	1.66	59.0	1.91
1.0	0.739	66.3	40.8	**68.3**	1.79	53.1	**2.25**

nism of §9.3 predicts: the threshold and magnitude effects come from the economics of prefill and restore, not from how predictable the workload is, so removing all per-user signal cannot remove them. Notably the gap is smallest exactly where LRU is strongest (σ=0, 95.3% hit rate), which is the regime least favorable to our argument and still not favorable enough to overturn it.

The sweep also sharpens a control reported in §8. Setting σ=0 does not drive AUC to chance but to 0.659, because recency, revisit count, and time-of-day remain informative even when users are interchangeable. The earlier i.i.d.-pool control, which reached AUC 0.56, destroyed user identity entirely and so removed more than per-user dispersion alone. The two measure different things, and σ is the more precise instrument: it isolates the per-user component while holding the rest of the feature set intact.

We do not claim this substitutes for observed return behavior. It bounds the sensitivity of our conclusions to one modeling choice, and shows that the conclusion which depends on that choice (whether learned eviction is worth building) is separable from the one that does not (whether hit rate should be the target).

10 Prototype Validation: TinyLlama

To validate with a real transformer, we integrated our `TieredCacheManager` with TinyLlama-1.1B-Chat (22 layers, 32 Q-heads, 4 KV-heads via GQA, head_dim=64) on CPU.

GQA Compatibility. Modern models use Grouped Query Attention where KV heads (4) differ from query heads (32). Our serialization must use `num_key_value_heads` for tensor dimensions, a detail that is easy to get wrong in production.

Metric definitions. We report TTFT and end-to-end speedups *separately*. Cold TTFT is a timed prefill-only forward pass, the cost a cache hit actually avoids; warm TTFT is tier load + tensor restore + one forward pass. A tempting shortcut, comparing a cold `generate()` total (prefill *plus* decode) against warm first-token latency, inflates the apparent speedup ∼2.5× by crediting the warm path with decode time both paths share; we flag this because the conflation is easy to make and appears in informal benchmarks. All GPU timings are bracketed by `cuda.synchronize()`.

Table 10: TinyLlama integration, 598-token prompt. CPU: FP32 serving, mean of 2 trials. T4 GPU: FP16 serving, median of 9 trials. GPU cold TTFT is derived as the median paired cold−warm E2E gap (prefill-only was not separately instrumented on the GPU runs).

Metric	CPU (FP32)	T4 GPU (FP16)
Cold TTFT (prefill)	4,783ms	∼37ms
Warm TTFT (load+restore+step)	251ms	∼40ms
TTFT speedup	**19.1**×	∼1×
E2E speedup (50 tok.)	1.43×	1.03×
Cache size (FP16)	13,156 KB	13,156 KB
Save / load latency	55 / 16ms	50 / 18ms
Byte-identical output	✓ (2/2)	✓ (9/9)

CPU results. Cold prefill of 4,783ms versus a 251ms warm path (16ms tier load + 25ms tensor restore + 210ms first forward pass) is a 19.1× TTFT improvement. End-to-end speedup is 1.43× because decode-phase speed is independent of cache state.

GPU results. On a T4 the same experiment collapses to parity: prefilling 598 tokens takes ∼37ms, comparable to the ∼40ms restore path (median E2E speedup 1.03× across 9 trials). This is what the break-even model (§11) predicts: 598 tokens sits *below* TinyLlama's crossover $N^* \approx 1{,}561$, where persistence should not pay on latency. §11 sweeps context length across the crossover.

Determinism: storage is exactly as faithful as memory. At 598 tokens every trial is byte-identical to the cold path on both devices. On short prompts (168 tokens), the resumed path deterministically diverges from the cold path at one near-tie token (both outputs coherent). An ablation attributes the cause: resuming from the *never-serialized, in-memory* cache reproduces the restored-cache output exactly, in every trial, so the storage round-trip (including FP16 quantization under FP32 serving) contributes *nothing*. The divergence stems entirely from computing the resumed token through incremental-decode kernels whose accumulation order differs from the cold path's batched prefill, which can flip near-tie argmax decisions. This is inherent to *any* cache-reuse scheme, including in-memory prefix caching in production servers; tiered persistence adds no divergence beyond it.

Cost model validation. These measurements are also

how we validate the value model that §9 depends on, and how we caught its predecessor. The cost model used throughout our earlier evaluation predicted **57,747ms** for this 598-token prefill against ∼37ms measured on the T4, an error of roughly three orders of magnitude, because it priced prefill with two hardcoded constants and no model geometry at all. The architecture-parameterized model of §8 predicts **47–71ms** for the same prefill at T4 specifications (peak 65 TFLOP/s, MFU 0.30–0.45), bracketing the measurement within a factor of two. Independently, its break-even lengths agree exactly with Table 11, which is computed from the same expressions in separate code. We regard agreement within 2× against hardware, plus internal consistency with the analytic model, as the minimum bar for a value model that policy conclusions rest on; the superseded model met neither, and no test in our suite would have caught it, because none existed.

11 A Break-Even Model for Restoration versus Recomputation

The 19.1× CPU TTFT speedup above evaporated to parity on a T4, where prefilling 598 tokens takes ∼37ms, comparable to the restore path itself. Persistence wins on raw latency only beyond a *break-even context length* N^*, which we derive analytically below; the GPU measurements land where the model predicts.

Restoring a persisted cache moves $O(N)$ bytes:

$$t_{\text{restore}}(N) = \ell_{\text{tier}} + \frac{2 \cdot L \cdot H_{kv} \cdot d \cdot b \cdot N}{B_{\text{tier}}} \tag{5}$$

where ℓ is tier latency, B bandwidth, L layers, H_{kv} KV heads, d head dimension, b bytes per element. Recomputing the prefill costs $O(N)$ weight FLOPs plus $O(N^2)$ attention:

$$t_{\text{prefill}}(N) = \frac{2PN + 4LH_q dN^2}{F \cdot \eta} \tag{6}$$

with P parameters, H_q *query* heads (GQA shrinks the cache, not the score computation), F peak FLOP/s, and η the achievable utilization. Since restore is linear and prefill superlinear, persistence always wins beyond the crossover N^*.

Table 11 and Figure 4 expose the governing quantity: the ratio of KV bytes per token to prefill FLOPs per token. For MHA models like Llama-2-7B, the KV cache is so large relative to compute that NVMe restore only beats prefill past ∼43k tokens; persistence is a *GPU-occupancy* optimization there, not a latency one. For large GQA models like Llama-2-70B, restore wins almost immediately (N^*=12 from NVMe):

Table 11: Break-even context length N^* (A100, η=0.45; NVMe: 10ms + 2GB/s; object storage: 100ms + 500MB/s). Persistence wins on latency beyond N^*.

Model	KV/token	N^* (NVMe)	N^* (S3)
TinyLlama-1.1B (GQA 4/32)	22 KB	1,561	25,901
Llama-2-7B (MHA 32/32)	512 KB	43,559	254,203
Llama-2-70B (GQA 8/64)	320 KB	12	289

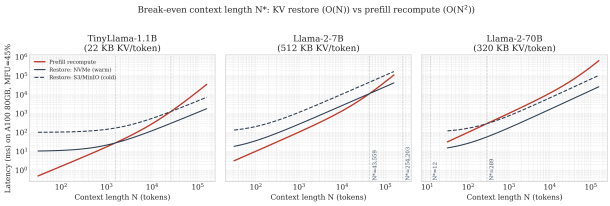


Figure 4: Prefill recompute ($O(N^2)$, red) vs. tier restore ($O(N)$, black) latency by context length. The crossover N^* varies by four orders of magnitude across model architectures.

grouped attention shrinks bytes moved while parameters inflate FLOPs avoided. As the industry converges on GQA/MQA architectures, tiered KV persistence shifts from "sometimes useful" to near-universally latency-positive. (Analysis and figure reproducible via `benchmarks/breakeven_analysis.py`.)

Empirical validation. Sweeping context length on a T4 (Table 12), the measured TTFT speedup rises monotonically from 0.62× at 168 tokens to 1.11× at 1,792 and crosses parity between 440 and 1,792, bracketing the analytic crossover (N^*≈1,561 with nominal NVMe constants). TinyLlama's 2,048-token context window caps that sweep just past its own crossover, so we repeat the measurement on Llama-3.2-1B (16 layers, GQA 8/32, 32 KB/token), whose 128k window allows a sweep entirely above N^*.

Three things follow. First, *the asymmetry the model rests on is directly visible*: over a 12× increase in context, cold prefill grows 14.8× while the warm path grows 1.8×. Fitting the warm measurements gives 94.5ms of fixed cost plus 13.4 μs per token, i.e. a linear transfer at an implied 2.44 GB/s against the 2.0 GB/s the model assumes. The bandwidth constant transfers; the fixed cost does not, because a Colab ephemeral disk read through `torch.load` is not an NVMe read.

Second, *restoration is faithful*. The maximum deviation between the warm and cold next-token logit vectors is 0.031 across every length, which is FP16 rounding on a logit scale spanning tens, and the argmax agrees in 25 of 25 trials. We report the logit deviation rather than argmax agreement alone because the latter is not sufficient: an earlier version of this experiment built the cache from the

Table 12: Measured TTFT crossover on a T4 (TinyLlama, 3–6 trials per length). Warm path = tier load + tensor restore + first forward pass. The speedup crosses 1× between 440 and 1,792 tokens, bracketing the analytic prediction.

Tokens	Cache	Cold TTFT	Warm path	TTFT speedup
168	3.7 MB	27ms	42ms	0.62×
440	9.7 MB	47ms	65ms	0.72×
1,792	39.4 MB	212ms	188ms	**1.11**×

Table 13: Long-context sweep, Llama-3.2-1B on a T4 (5 trials per length, medians). The cache covers tokens $0..N-2$, so the warm path reconstructs exactly the cold path's state; `max Δlogit` is the deviation between the two next-token logit vectors.

Tokens	KV	Cold	Warm	Speedup	max Δlogit
512	16.8 MB	111ms	101ms	1.09×	0.020
1,024	33.5 MB	157ms	115ms	1.36×	0.023
2,048	67.1 MB	340ms	115ms	2.97×	0.031
4,096	134.2 MB	878ms	146ms	6.02×	0.027
6,144	201.3 MB	1,645ms	181ms	**9.11**×	0.027

full prompt and re-fed the final token, duplicating it, and argmax agreement read 25 of 25 while the logit deviation was 7.9 to 16.4. Only the finer check exposed it.

Third, *the model's shape holds but its constants are platform-specific*. Measured prefill runs 1.15–1.66× slower than predicted at the assumed MFU of 0.30, implying an effective MFU near 0.22 on this GPU. The measured parity crossing sits near 500 tokens rather than the N^*=91 that nominal NVMe constants predict, and the gap is dominated by the 85ms of extra fixed storage cost rather than by the compute model: at 91 tokens the modeled restore is 11ms, while the measured path costs 96ms. We therefore read this sweep as confirming the functional form, that prefill is superlinear, restore is linear, and a crossover exists above which persistence pays, while treating N^* itself as a quantity that must be computed from the storage path actually deployed.

12 Discussion & Future Work

The Metric Mismatch. Hit rate assumes uniform item value. KV caching violates that assumption twice over: a hit below N^* is worth literally zero, and above it savings grow superlinearly. The consequence is not a loose correlation but a reversal, with the hit-rate winner delivering the least value on our enterprise workload. Research and production systems should report value-weighted metrics (GPU-seconds saved per decision, discounted by restore cost); hit rate remains meaningful only after showing that cached items are near-uniform in value, which is a property of the workload and the served architecture, not something a policy can assume.

Why does LRU win on hit rate, and lose on value? It wins the first because our workloads have strong temporal locality: most resumes occur within an hour of the previous access, which is exactly LRU's inductive bias, obtained without training anything. It loses the second because the same blindness that makes it robust also makes it indiscriminate. Consuming only access order, LRU cannot tell a 512-token session from an 8k one, nor a hot-tier hit from a cold-tier hit that costs more to restore than to recompute; on enterprise traffic it earns 812 of its 1,237 hits in a tier where every hit is worth zero. Robustness and precision are the same property viewed from two sides, and which one matters depends entirely on whether hits are uniform in value.

A caution on baselines. LRU's insensitivity has a methodological consequence. It consumes one input, access order, while learned policies consume nine features whose scales must match training, and value-aware policies additionally consume a cost model. A defect in any of those inputs degrades the richer policy and leaves LRU untouched, so evaluation errors do not perturb rankings symmetrically. Both defects we found in our own pipeline behaved this way (§9.1). We suspect published comparisons against LRU that do not control replay-time semantics and cost-model fidelity deserve re-examination in both directions: wins for learned eviction may be artifacts, and so may losses.

A decision rule, in order of payoff. (1) Compute N^* for the served architecture at each tier (§11). It is closed-form, needs no measurement beyond published geometry and storage constants, and determines everything below. (2) *Do not build tiers whose N^* exceeds the workload's context lengths.* This is the largest lever we measured: 3.6× (§9.5), against 1.99× for the best policy choice inside a badly provisioned hierarchy. (3) Refuse admission to entries below the surviving tiers' N^*; this is arithmetic on the entry's token count, not a prediction, and it recovers most of what a value-aware policy would. (4) Only then choose a policy. If context lengths sit far above N^*, hits are near-uniform in value, hit rate is an adequate proxy, and LRU is free and hard to beat. If the distribution straddles N^*, value-aware eviction adds a further 1.15×; static value density suffices, and lifetime normalization earns its keep only under high size variance (σ=1.2). (5) Never tune on hit rate without first establishing which regime applies.

Limitations. (1) The real-trace evaluation replays production arrivals and sizes, but session *return behavior* remains modeled. §9.9 bounds how much this matters by sweeping the parameter that controls it, and finds the metric conclusion stable across the range, but a bound is not

a measurement: public LLM inference traces carry no conversation identifiers, and we show linkage is not reconstructable from them (§8.1). Datasets with hashed conversation identifiers (e.g., WildChat, LMSYS-Chat-1M) could close this gap. (2) GPU validation used a single T4. The Llama-3.2-1B sweep covers the region well above N^*, but server-class GPUs (A100/H100, where higher FLOPs push N^* further out) remain unmeasured, as does a production storage path: our measured fixed restore cost is an order of magnitude above a real NVMe, which moves the empirical crossover accordingly. (3) Tier restore costs are modeled, not measured on production hardware. (4) The simulation stores a downscaled KV geometry as a scale model: capacity dynamics run on 512-byte-per-token entries while value and restore are priced at the served architecture, so absolute occupancy is not physical even though the session-count-to-capacity ratio and the economics are. (5) Value is modeled throughout, from measured hardware constants but not from an instrumented production server; the absolute GPU-seconds should be read as the output of a validated model (§10) rather than as a measurement.

What remains open. Value-aware eviction already beats LRU on value where the workload straddles N^*, but no policy we built wins *both* metrics anywhere, and the most promising design would not have to: a hybrid that follows recency by default and deviates only for entries in the value tail should capture LRU's locality while avoiding its indiscriminate cold-tier hits. Two further directions follow from our results. Tier-aware admission, refusing to promote entries whose restore cost exceeds their recompute value, computable in closed form from N^*, should recover most of LRU's wasted hits at no prediction cost. And because our value model makes cold-tier hits worthless for small models, the tier hierarchy itself should be architecture-dependent rather than fixed.

13 Conclusion

We set out to beat LRU at cache eviction with a learned policy, and failed: across three synthetic workloads, ten paired seeds, two model architectures, and a replay of the Azure LLM inference production trace, LRU wins cache hit rate every time.

That verdict, however, does not mean what it appears to mean. Scored by GPU time saved, the same LRU that wins every hit-rate comparison delivers the least value of any policy on our enterprise workload, where a value-density policy conceding 21 hit-rate points saves 1.99× as much compute, and still 1.15× once we correct a provisioning error that the same analysis exposes. Hit rate treats all hits as equal; KV caching makes them unequal twice over, since a hit below the break-even length N^* is worth nothing at all and savings above it grow superlinearly with context. Where those effects are absent (workloads far above N^*, or large grouped-query models where N^* falls to tens of tokens), the metrics agree and LRU is genuinely the right choice. Where the workload straddles N^*, optimizing hit rate selects the worst policy available and reports it as the best.

The clearest evidence is an ablation in which everything else is removed: two clairvoyant policies, both caching perfectly at a 100.0% hit rate, still differ significantly in delivered value, because the tier an entry sits in determines what its hit is worth and hit rate cannot represent that. We arrived here by correcting our own evaluation twice, and the corrections moved in opposite directions: one flattered the learned policy, the other flattered LRU and had supported a collapse-of-value-density finding that did not survive re-pricing. Because a recency policy consumes strictly less information than its competitors, it is strictly less exposed to defects in that information, and evaluation errors consequently favor whichever side depends on the broken part. We think that asymmetry, more than any individual number in this paper, is what the learned-caching literature should take from it.

The practical consequence is not a better policy but a cheaper one. Both effects that hit rate misses are closed-form in N^*, so most of what a value-aware policy buys is available as arithmetic: refuse to build tiers that cannot repay their restore, refuse admission to entries below threshold, and let recency do the rest. Deleting one such tier was worth 3.6× in our measurements, more than every eviction policy we built. Beating LRU on both metrics at once remains open; beating the metric does not require beating LRU.

References

[1] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. LHD: Improving cache hit rate by maximizing hit density. In *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2018.

[2] László A. Bélády. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.

[3] Pei Cao and Sandy Irani. Cost-aware WWW proxy caching algorithms. In *Proceedings of the USENIX Symposium on Internet Technologies and Systems (USITS)*, 1997.

[4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness.

In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022.

[5] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *Proceedings of the 2024 USENIX Annual Technical Conference (ATC)*, 2024.

[6] In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of the 7th Annual Conference on Machine Learning and Systems (MLSys)*, 2024.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[8] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[9] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024.

[10] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[11] LMCache Team. LMCache: Accelerating LLM serving with reusable KV caches. GitHub Repository, 2024. URL https://github.com/LMCache/LMCache.

[12] Microsoft Azure. AzurePublicDataset: Azure LLM inference trace 2024. GitHub Repository, 2024. URL https://github.com/Azure/AzurePublicDataset.

[13] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025.

[14] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[15] Yakov Pyotr Shkolnikov. Agent memory below the prompt: Persistent Q4 KV cache for multi-agent LLM inference on edge devices. arXiv preprint arXiv:2603.04428, 2026.

[16] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. Learning relaxed Béládys for content distribution network caching. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.

[17] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.

[18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[19] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.

[20] Neal Young. On-line file caching. In *Proceedings of the 5th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 82–86, 1994.

[21] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.